

Programming and Proving in Lean

Lecture 2 of 4

Leo de Moura

Senior Principal Applied Scientist, AWS

Chief Architect, Lean FRO

Marktoberdorf Summer School | August 2026



Recap and Plan

From Lecture 1: every term has a type; propositions are types; proofs are programs; $\wedge$, $\vee$, $\neg$, $\exists$ are ordinary definitions, not built-ins; tactics construct proof terms, and the kernel checks them.

Today we meet the machinery those definitions were built with — and start proving things about **programs**:

1. **Inductive types**: where `Nat`, `List`, and `Or` all come from.
2. **Structures and type classes**: how Lean organizes interfaces.
3. **Lean as a programming language**: monads, `IO`, performance.
4. **Build a language**: expressions, an evaluator, an optimizer.
5. **Prove the optimizer correct** — four times, with less and less work.



Inductive Types: Enumerations

```
inductive Bool where
| false
| true

def and : Bool → Bool → Bool
| .true, b => b
| .false, _ => .false
```

- An inductive type is **freely generated by its constructors**: these are all the values, and they are distinct.
- Functions out of it: **pattern matching**.
- This is the actual definition of **Bool** — the core library starts from inductive types, not primitives.



Recursive Types: `Nat`

```
inductive Nat where
| zero
| succ (n : Nat)

def add : Nat → Nat → Nat
| m, .zero    => m
| m, .succ n => .succ (add m n)
```

- Constructors may take the type itself as an argument:
`Nat` is zero and the successors — unary numbers.
- Literals like `5` are notation; fast arithmetic comes from an efficient runtime representation, with the inductive definition as its specification.
- `add` is defined by **structural recursion** on the second argument.



Lists

```
inductive List (α : Type) where
| nil
| cons (head : α) (tail : List α)

def append : List α → List α → List α
| .nil,      ys => ys
| .cons x xs, ys => .cons x (append xs ys)
```

- Inductive types can be **parameterized**: `List Nat` , `List Bool` , ...
- `[1, 2, 3]` and `xs ++ ys` are notation for `cons` / `append` .
- Same three ingredients every time: constructors, pattern matching, structural recursion.



Option : Making Failure Explicit

```
inductive Option (α : Type) where
| none
| some (a : α)
```

```
def first? : List α → Option α
| []      => none
| x :: _ => some x
```

```
#eval first? [1, 2, 3]
```

```
{ some 1
```

```
#eval first? ([] : List Nat)
```

```
{ none
```

- The type checker forces every caller to handle **none** .



Pattern Matching

```
def fib : Nat → Nat
| 0      => 0
| 1      => 1
| n + 2 => fib (n + 1) + fib n

#eval fib 10
55

example : fib 7 = 13 := rfl

def describe : List Nat → String
| []      => "empty"
| [x]     => s!"one element: {x}"
| x :: _ :: _ => s!"starts with {x}"

#eval describe [5, 7, 2]
"starts with 5"
```

- Patterns nest; $n + 2$ is a pattern (two successors); wildcards, literals, and overlapping cases (first match wins) all work.
- **match** is not primitive: Lean compiles it to **recursors** — everything reduces to a small trusted core.



Termination by Construction

```
def len : List α → Nat
| []      => 0
| _ :: xs => len xs + 1
```

- Every Lean function **terminates** — otherwise proofs would be unsound: a nonterminating **loop : False** would "prove" anything.
- Structural recursion (recursive calls on subterms) is checked automatically. **len** , **add** , **fib** : all structural.
- Termination checking is behind every **def** you have seen so far, invisibly.



Beyond Structural Recursion

```
def gcd' (a b : Nat) : Nat :=  
  if h : a = 0 then b  
  else gcd' (b % a) a  
termination_by a  
decreasing_by exact Nat.mod_lt _ (Nat.pos_of_ne_zero h)  
  
#guard gcd' 12 20 = 4
```

- `gcd'` does not recurse on a subterm — but **a decreases**.
- `termination_by` names the measure;
`decreasing_by` proves it decreases. Often Lean finds both automatically; here it needs the hint.
- `if h : a = 0` — the **dependent** if: the branch gets a proof `h`.



When Termination Is Hard

```
partial def collatzSteps (n : Nat) : Nat :=  
  if n ≤ 1 then 0  
  else if n % 2 == 0 then 1 + collatzSteps (n / 2)  
  else 1 + collatzSteps (3 * n + 1)  
  
#eval collatzSteps 27  
| 111
```

- **partial def** : opt out of termination checking. You can **run** it, but you cannot **unfold** it in proofs — soundness is preserved.
- Middle grounds: an explicit fuel parameter (Lecture 4's interpreter), or **partial_fixpoint** — partial correctness reasoning for possibly-nonterminating functions.



Structures

```

structure Point where
  x : Int
  y : Int
  deriving Repr

def p : Point := { x := 1, y := 2 }

def moveRight (q : Point) : Point := { q with x := q.x + 1 }

def Point.normSq (q : Point) : Int := q.x * q.x + q.y * q.y

#eval (moveRight p).normSq
| 8

```

- A structure is an inductive type with one constructor and named fields. `{ q with ... }` builds an updated copy — no mutation, and (as we will see) often no copy either.
- Declarations live in **namespaces**, and **dot notation** finds them from the type: `p.normSq` is `Point.normSq p` — likewise `xs.map f`, `e.eval σ`.
- `deriving Repr` will be explained later.



The Overloading Problem

```
#check (2 : Nat) + 2
| 2 + 2 : Nat
#check (2.0 : Float) + 2.0
| 2.0 + 2.0 : Float
```

- `+` works for **Nat** , **Int** , **Float** , **BitVec 32** , matrices, ... Different code for each. Who picks?
- We also want to write functions that work for **any** type that has `+` .
- The mechanism: **type classes** — interfaces whose implementations are found by the elaborator.



Type Classes

```
class Add (α : Type u) where
  add : α → α → α

instance : Add Point where
  add p q := { x := p.x + q.x, y := p.y + q.y }

def a : Point := { x := 1, y := 2 }

#eval a + a
| { x := 2, y := 4 }
def sum3 [Add α] (a b c : α) : α := a + b + c

#eval sum3 a a a
| { x := 3, y := 6 }
#eval sum3 2.0 1.0 3.0
| 6.000000
#eval sum3 1 (-2) 3
| 2
```

- **Add** is a structure; an **instance** is a value of it, registered for **instance search**.
- **[Add α]** — an implicit argument found by searching registered instances. **sum3** works for every type with **+**.



Instances Compose

```

structure Pair (α : Type) where
  fst : α
  snd : α
  deriving Repr

instance [Add α] : Add (Pair α) where
  add u v := ⟨u.fst + v.fst, u.snd + v.snd⟩

#eval Pair.mk 1 2 + Pair.mk 30 40
{ fst := 31, snd := 42 }
def b : Point := { x := 3, y := 1 }
def c : Point := { x := 0, y := 4 }
#eval Pair.mk b c + Pair.mk c b
{ fst := { x := 3, y := 5 }, snd := { x := 3, y := 5 } }

```

- Instances can **require** instances: if α has $+$, so does **Pair** α .
- Instance search chains them: **Add (Pair Point)** is found from **Add Point**, which we wrote by hand.
- Mathlib runs on this mechanism: 1,500+ classes, 20,000+ instances.



Classes You Meet Every Day

```
instance : ToString Point where
  toString q := s! "{q.x}, {q.y}"

#eval toString p
| "(1, 2)"
#eval s! "the point is {p}"
| "the point is (1, 2)"
```

- **OfNat** — numeric literals; `2` means `OfNat.ofNat 2` at any type.
- **BEq**, **DecidableEq** — equality tests; **#guard** uses them.
- **Repr**, **ToString** — printing; **#eval** uses them.
- **GetElem** — the `xs[i]` and `xs[i]?` notation.
- In Lecture 3: **grind** activates theory solvers through type classes.



Deriving Instances

```
inductive Color where
| red
| green
| blue
deriving Repr, BEq, DecidableEq

#eval Color.red
| Color.red
#guard Color.red != Color.blue
```

- **deriving** generates boilerplate: printing (**Repr**), boolean equality (**BEq**), decidable propositional equality (**DecidableEq**), ...



Classes with proof fields

```
class Semigroup (α : Type) where
  op : α → α → α
  assoc : ∀ a b c, op (op a b) c = op a (op b c)

instance : Semigroup Nat where
  op := (· + ·)
  assoc := Nat.add_assoc

example : Semigroup.op 2 3 = 5 :=
  rfl

-- Generic code using the proof field
open Semigroup in
example [Semigroup α] (a b c : α) : op a (op b c) = op (op a b) c :=
  (assoc a b c).symm
```

- Classes can carry **laws**: **assoc** is a proof field, and every instance must prove it.
- Generic code gets the laws for free — this is how Mathlib's algebraic hierarchy works, and how **grind** finds its theory solvers (Lecture 3).



Chaining Options: the Pyramid of Doom

```
def firstThird (xs : List  $\alpha$ ) : Option ( $\alpha \times \alpha$ ) :=  
  match xs[0]? with  
  | none => none  
  | some a =>  
    match xs[2]? with  
    | none => none  
    | some c => some (a, c)
```

- Every step can fail; every step repeats the same plumbing.
- The pattern — sequence computations, propagate failure — is a **monad**. **Option** , **IO** , **StateM** , parsers: same shape.



Monads: Write the Plumbing Once

```
class Monad (m : Type u → Type v) where
  pure :  $\alpha \rightarrow m \alpha$ 
  bind :  $m \alpha \rightarrow (\alpha \rightarrow m \beta) \rightarrow m \beta$ 
```

```
instance : Monad Option where
  pure := some
  bind x f :=
    match x with
    | none   => none
    | some a => f a
```

```
def firstThird (xs : List  $\alpha$ ) : Option ( $\alpha \times \alpha$ ) :=
  xs[0]? >>= fun a =>
  xs[2]? >>= fun c =>
  pure (a, c)
```

- $x \gg= f$ is notation for `bind x f`; `pure` wraps a value.
- The `none` check now lives in one place: `bind`.
`firstThird` never mentions failure at all.



do -Notation

```
def firstThird (xs : List  $\alpha$ ) : Option ( $\alpha \times \alpha$ ) := do
  let a ← xs[0]?
  let c ← xs[2]?
  return (a, c)

#eval firstThird [10, 20, 30, 40]
| some (10, 30)
#eval firstThird [10]
| none
```

- **do** works for **any** monad; the **Monad** type class provides the plumbing.
- **let** $x \leftarrow e$ sequences; failure short-circuits automatically.



StateM : State as an Effect

```
def StateM (S α : Type) := S → α × S

instance : Monad (StateM S) where
  pure a := fun s => (a, s)
  bind x f := fun s =>
    let (a, s') := x s
    f a s'

def get : StateM S S := fun s => (s, s)

def set (s' : S) : StateM S Unit := fun _ => ((), s')
```

- A stateful computation is a **pure function**: it takes the initial state and returns the result together with the final state.
- **pure a** leaves the state alone. **bind** runs **x**, then feeds the result **and the updated state** to **f** — the plumbing **Option**'s **bind** wrote once for failure, written once for state.
- **get** and **set** are one-liners once the representation is visible.



Programming with **StateM**

```
def double : StateM Nat Unit := do
  let s ← get
  set (s + s)

example : StateM Nat Unit = (Nat → Unit × Nat) := rfl

#eval double.run 21
[ ((), 42)
```

- The same **do** , a different effect: one mutable state. **get** reads it, **set** writes it, **modify f** applies **f** to it.
- The **example** is **rfl** : the library's **StateM** really is the function type from the previous slide (generalized via **StateT**); same **pure** , same **bind** .
- Running is function application: **double.run 21** yields **((), 42)** .
- Lecture 4 verifies exactly these programs: Hoare triples over **StateM** , by symbolic execution.



I0

```
def greet : IO Unit := do
  IO.println "Marktoberdorf!"
  let ms ← IO.monoMsNow
  IO.println s!"clock: {ms} ms"
```

```
#eval greet
Marktoberdorf!
clock: 5871428685 ms
```

- Side effects live in the **IO** monad — visible in the type, sequenced with the same **do** .
- A Lean program is a **main : IO Unit** ; **lake** builds native executables.
- Lean's compiler, language server, and build system are Lean programs.

Building a Language

Syntax trees, an evaluator, an optimizer — and then we prove things.



Syntax Trees

```
inductive Expr where
| const (n : Int)
| var   (x : String)
| add   (a b : Expr)
| mul   (a b : Expr)
deriving Repr, BEq
```

```
#eval Expr.add (.var "x") (.const 3)
Expr.add (Expr.var "x") (Expr.const 3)
```

- An arithmetic expression language: literals, variables, $+$, $*$.
- Inductive types are **made** for syntax trees — this is the seed of Lecture 4, where statements, semantics, and a verifier grow around it.



The State

```
def State := String → Int

def State.init : State := fun _ => 0

def State.get (σ : State) (x : String) : Int := σ x

def State.set (σ : State) (x : String) (v : Int) : State :=
  fun y => if y = x then v else σ y

#eval ((State.init.set "x" 3).set "y" 7).get "x"
| 3
```

- A program state is a **total map** from variable names to values: **.get** reads, **.set** updates by shadowing. Total maps mean no error cases to thread through proofs.
- This is not **StateM**'s **get / set**: this **State** is the **value** a stateful program manipulates — in **StateM** **State** α , this type would be the **S**.
- This small API carries the rest of the course: Lecture 3 teaches **grind** its two laws, and Lecture 4's verifier runs on it.



Evaluation

```
def Expr.eval (σ : State) : Expr → Int
| .const n => n
| .var x   => σ.get x
| .add a b => a.eval σ + b.eval σ
| .mul a b => a.eval σ * b.eval σ

#eval (Expr.add (.var "x") (.const 3)).eval (State.init.set "x" 10)
13
```

- The evaluator is ten lines. This is a **definitional interpreter**: the semantics of the language, as a program.
- Variables read through **State.get** ; everything else is arithmetic on recursive results.



Concrete Syntax via Macros

```
syntax "[Expr]" term " " : term
```

```
macro_rules
```

```
| `([Expr| $n:num])    => `(Expr.const $n)
| `([Expr| $x:ident]) => `(Expr.var $(Lean.quote x.getId.toString))
| `([Expr| ($e)])      => `([Expr| $e])
| `([Expr| $a + $b])   => `(Expr.add [Expr| $a] [Expr| $b])
| `([Expr| $a * $b])   => `(Expr.mul [Expr| $a] [Expr| $b])
```

```
syntax:max term "[ " ident "]" : term
```

```
syntax:max term "[ " ident " := " term "]" : term
```

```
macro_rules
```

```
| `($σ[$x])          => `(State.get $σ $(Lean.quote x.getId.toString))
| `($σ[$x := $v])    => `(State.set $σ $(Lean.quote x.getId.toString) $v)
```

```
#eval [Expr| x + 3 * y].eval (State.init[x := 2][y := 4])
```

```
| 14
```

- Five macro rules reuse Lean's parser for our language.
- The same mechanism gives the **State** API its notation:
 $\sigma[x]$ expands to $\sigma.\text{get } "x"$, and $\sigma[x := v]$ to $\sigma.\text{set } "x" \ v$. Lectures 3 and 4 use both.



Constant Folding with Smart Constructors

```
def mkAdd : Expr → Expr → Expr
| .const 0, b => b
| a, .const 0 => a
| a, b => .add a b

def mkMul : Expr → Expr → Expr
| .const 1, b => b
| a, .const 1 => a
| .const 0, _ => .const 0
| _, .const 0 => .const 0
| a, b => .mul a b

def Expr.optimize : Expr → Expr
| .const n => .const n
| .var x    => .var x
| .add a b  => mkAdd a.optimize b.optimize
| .mul a b  => mkMul a.optimize b.optimize
```

- Recurse first, then fix up with a **smart constructor** per operation.



Test First: **#guard**

```
#guard [Expr| 0 + x].optimize == [Expr| x]
#guard [Expr| 1 * (x + 0)].optimize == [Expr| x]
#guard [Expr| 0 * (x + y)].optimize == [Expr| 0]
#guard [Expr| x + 2 * 3].optimize == [Expr| x + 2 * 3]
```

- **#guard** evaluates at compile time; the file does not build if a test fails.
- Tests catch bugs early and cheaply. But **optimize** has infinitely many inputs — for **every input**, we need a theorem.

From Tests to Theorems

[Resume presentation](#)

Induction: the Anatomy

```
theorem len_append (xs ys : List α) :  
  len (xs ++ ys) = len xs + len ys := by  
  induction xs with  
  | nil => simp [len]  
  | cons x xs ih => simp [len, ih]; lia
```

- **induction xs** : one goal per constructor; the **cons** goal gets the **induction hypothesis ih** for the tail.
- Compare Lecture 1's case analysis: **induction** is **cases** plus hypotheses for the recursive arguments.
- **simp [len]** unfolds by the defining equations; **lia** finishes arithmetic.



A Development Is a Lemma Chain

```
def rev : List α → List α
| []      => []
| x :: xs => rev xs ++ [x]

theorem rev_append (xs ys : List α) :
  rev (xs ++ ys) = rev ys ++ rev xs := by
  induction xs with
  | nil => simp [rev]
  | cons x xs ih => simp [rev, ih]

theorem rev_rev (xs : List α) : rev (rev xs) = xs := by
  induction xs with
  | nil => rfl
  | cons x xs ih => simp [rev, rev_append, ih]
```

- `rev_rev` needs `rev_append` (which needs `++` lemmas, provided by the library).



Designing `simp` Lemmas

```

theorem mkAdd_eval (a b : Expr) (σ : State) :
  (mkAdd a b).eval σ = a.eval σ + b.eval σ := by
  unfold mkAdd
  split
  · simp [Expr.eval]
  · simp [Expr.eval]
  · simp [Expr.eval]

-- Same proof using `<=>`
example (a b : Expr) (σ : State) :
  (mkAdd a b).eval σ = a.eval σ + b.eval σ := by
  unfold mkAdd
  split <=> simp [Expr.eval]

theorem mkMul_eval (a b : Expr) (σ : State) :
  (mkMul a b).eval σ = a.eval σ * b.eval σ := by
  unfold mkMul
  split <=> simp [Expr.eval]

```

- One lemma per helper function, stating "the smart constructor means what the plain constructor means".
- Good **`simp`** lemmas rewrite toward a **normal form** — here: plain arithmetic on evaluations.
- **`split`** performs the case analysis of the **`match ; <=>`** applies the next tactic to every resulting goal.



Arithmetic: **lia** and **calc**

```
example (a b : Nat) (h1 : a ≤ b) (h2 : b ≤ a) : a = b := by  
  lia  
  
example (a b c d : Nat) (h1 : a = b) (h2 : b ≤ c) (h3 : c + 1 < d) :  
  a < d := by  
  calc a = b := h1  
    _ ≤ c := h2  
    _ < d := by lia
```

- **lia** : decision procedure for linear integer arithmetic. Use it freely; do not prove $a + 1 \leq b + 1$ by hand.
- **calc** : chains of equalities and inequalities, written the way you would on paper.



Proving the Optimizer Correct

```
theorem optimize_correct (e : Expr) (σ : State)
  : e.optimize.eval σ = e.eval σ := by
  induction e with
  | const n => rfl
  | var x   => rfl
  | add a b iha ihb => simp [Expr.optimize, mkAdd_eval, Expr.eval, iha, :
  | mul a b iha ihb => simp [Expr.optimize, mkMul_eval, Expr.eval, iha, :
```

- With the helper lemmas in place, every case is one **simp**.
- Without them: a nested case analysis in each branch — try it.
- The lesson is about **structuring definitions and lemmas**, not about tactic tricks. This habit is most of proof engineering.



fun_induction : Follow the Definition

```
theorem optimize_correct2 (e : Expr) (σ : State) :  
  e.optimize.eval σ = e.eval σ := by  
  fun_induction Expr.optimize <;>  
  simp_all [Expr.eval, mkAdd_eval, mkMul_eval]
```

- **induction** `e` follows the **type**; **fun_induction** **optimize** follows the **definition** — one case per equation, match already split, induction hypotheses for each recursive call.
- Proofs about a function should follow the function.



The Ladder

```
theorem optimize_correct (e : Expr) (σ : State) :
  e.optimize.eval σ = e.eval σ := by
  fun_induction Expr.optimize <;> grind [Expr.eval, mkAdd.eq_def, mkMul.e
```

Same statement, decreasing effort:

1. **induction** + **split** + **simp** + **lia** — everything by hand (this one is yours, in the exercises).
2. helper lemmas + **induction**
3. **fun_induction** <;> **simp_all** — the induction follows the definition.
4. **fun_induction** <;> **grind** — automation handles the cases.

Lecture 3 is about what is inside **grind**.



A Harder One: Idempotence

```
theorem optimize_idempotent (e : Expr) :  
  e.optimize.optimize = e.optimize := by  
  fun_induction Expr.optimize <=>  
    grind [Expr.optimize, mkAdd.eq_def, mkMul.eq_def]
```

- Optimizing twice changes nothing — needs case analysis on what the smart constructors **returned**.
- `mkAdd.eq_def` hands `grind` the full match equation so it can split on the match arms itself.
- A manual proof is long and fiddly; the exercises let you try.



Verifying an Optimized Implementation

```
def sumTo : Nat → Nat
| 0 => 0
| n + 1 => sumTo n + (n + 1)

def sumToTR (n : Nat) : Nat := go n 0
where
  go : Nat → Nat → Nat
  | 0, acc => acc
  | n + 1, acc => go n (acc + (n + 1))

theorem sumToTR_go (n acc : Nat) : sumToTR.go n acc = sumTo n + acc := by
  induction n generalizing acc with
  | zero => simp [sumToTR.go, sumTo]
  | succ n ih => simp [sumToTR.go, sumTo, ih]; omega

theorem sumToTR_eq (n : Nat) : sumToTR n = sumTo n := by
  simp [sumToTR, sumToTR_go]
```

- The recursive call changes **acc** , so the lemma must **generalize** it: `induction n generalizing acc` .



Exercises

`Exercises/Lecture2.lean` :

1. `MyList` : `append` , `map` , `reverse` , and the classic lemmas (`length_append` , `map_map` , `reverse_reverse` — build the chain).
2. `Vec2` with `Add` and `ToString` instances; prove `add_comm` .
3. `StateM` : `tick` and `swapPair` — make the `#guard` specs pass.
4. The `State` API laws (`get_set_same` , `get_set_ne`), then the optimizer: implement it, prove it correct, (**harder**) prove idempotence.
5. `sumTo` vs. its tail-recursive version — generalize the invariant.

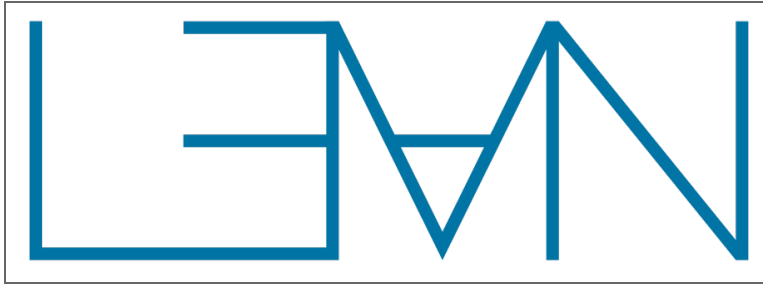
Solutions with notes in `Solutions/` .



Summary

- Inductive types + pattern matching + structural recursion: the whole language, including its logic, is built from them.
- Type classes organize interfaces; instance search composes them.
- Monads and **do** : effects with types — **Option** , **StateM** , **IO** .
- Proof engineering: helper lemmas, **fun_induction** , generalize the invariant — design beats tactics.

Next lecture: inside the automation — **simp** , **grind** , **bv_decide** — and what AI does with it.



Thank You

Leo de Moura

lean-lang.org

leanprover.zulipchat.com

leodemoura.github.io

Marktoberdorf Summer School | August 2026

Speaker notes